

Jobsheet 3 (Versi Bootstrap) — Responsive Design dengan Framework

Sub-CPMK: Membangun tampilan responsif memakai framework CSS (Bootstrap 5).

Versi ini adalah alternatif dari [jobsheet-03](#) (CSS murni) — halaman dan fungsionalitasnya identik, tapi seluruh tata letak dibangun memakai Bootstrap 5 (dimuat via CDN) alih-alih menulis CSS dari nol.

Perubahan dari Jobsheet 3 (CSS Murni)

- Bootstrap 5.3 dimuat via CDN (`bootstrap.min.css` + `bootstrap.bundle.min.js`).
- Navbar: hamburger memakai komponen `.navbar` / `.navbar-toggler` / `.collapse` bawaan Bootstrap (butuh JavaScript), menggantikan checkbox hack murni CSS.
- Kartu statistik: grid CSS manual diganti sistem grid 12 kolom Bootstrap (`.row` / `.col-md-4`).
- Section dibungkus komponen `.card` menggantikan styling `<section>` custom.
- Tabel & form memakai utility class Bootstrap (`.table-striped`, `.table-hover`, `.form-control`, `.btn-warning`, dst).
- `assets/css/style.css` menyusut dari ~245 baris menjadi ~15 baris (hanya override warna brand).

Cara menjalankan

Buka `index.html` di browser (butuh koneksi internet karena Bootstrap dimuat dari CDN), uji dengan DevTools responsive mode pada breakpoint Bootstrap (mobile <768px, tablet ~768–991px, desktop ≥992px).

Dokumentasi

Penjelasan lengkap tiap perubahan ada di folder [Dokumentasi/](#), termasuk tabel perbandingan class Bootstrap vs CSS murni.

Catatan

- Warna brand SIMPUS-Mini (`#1d5b8a`) bukan bagian tema bawaan Bootstrap, sehingga tetap ditulis manual lewat atribut `style` dan sedikit CSS custom — lihat [Dokumentasi bab 6 §6.3](#).

Dokumentasi Jobsheet 3 (Versi Bootstrap)

Dokumentasi ini adalah **versi alternatif** dari [dokumentasi jobsheet-03](#) yang memakai CSS murni (hand-written CSS). Fungsionalitas dan halamannya **identik** — Beranda, Daftar Buku, Tambah Buku, Daftar Anggota, Tambah Anggota — tapi seluruh tata letak dan komponen visualnya dibangun ulang memakai **framework CSS Bootstrap 5**, bukan CSS custom dari nol.

Kalau kamu belum paham konsep responsive design dasar (viewport, media query, breakpoint), baca dulu [bab 1 dokumentasi jobsheet-03 asli](#) karena dokumentasi ini akan sering membandingkan "cara CSS murni" vs "cara Bootstrap" untuk menyelesaikan masalah yang sama.

Kenapa Ada Dua Versi?

Tujuannya supaya kamu bisa membandingkan langsung **dua pendekatan** untuk mencapai hasil visual yang mirip:

	Jobsheet 3 (CSS Murni)	Jobsheet 3 (Bootstrap)
Layout	<code>display: flex</code> , <code>display: grid</code> ditulis manual	<code>.row</code> / <code>.col-*</code> (grid 12 kolom Bootstrap)
Navbar & hamburger	Checkbox hack (<code>:checked</code> + sibling combinator <code>~</code>)	Komponen <code>.navbar</code> bawaan + sedikit JavaScript Bootstrap
Kartu	<code><section></code> + CSS custom (<code>border-radius</code> , <code>box-shadow</code> manual)	Komponen <code>.card</code> bawaan
Tabel	<code><table></code> + CSS custom (<code>nth-child</code> , <code>:hover</code>)	Class utility <code>.table</code> , <code>.table-striped</code> , <code>.table-hover</code>
Form	<code><input></code> / <code><select></code> + CSS custom	Class utility <code>.form-control</code> , <code>.form-select</code> , <code>.form-label</code>
Breakpoint	Ditulis sendiri (<code>768px</code> , <code>480px</code>)	Bawaan Bootstrap (<code>sm</code> , <code>md</code> , <code>lg</code> , <code>xl</code> , <code>xxl</code>)
Total baris CSS custom	~245 baris (<code>style.css</code>)	~15 baris (<code>style.css</code>)

Intinya: **hasil akhirnya bisa mirip, tapi cara mencapainya sangat** berbeda.** CSS murni memberi kontrol penuh tapi butuh menulis semua** aturan sendiri; Bootstrap memberi banyak komponen & utility class siap pakai, dengan konsekuensi harus memuat file CSS/JS tambahan dan mengikuti konvensi nama class-nya.

Daftar Isi

1. [Konsep Dasar Bootstrap](#)
2. [Apa yang Berubah di File HTML?](#)
3. [Navbar Responsif ala Bootstrap](#)
4. [Grid System & Komponen Card](#)
5. [Tabel & Form dengan Utility Class Bootstrap](#)
6. [Rangkuman & Perbandingan dengan CSS Murni](#)

Struktur Folder

```
jobsheet-03-bootstrap/
├── index.html           # Beranda
├── assets/
│   └── css/
│       └── style.css    # Override kecil di atas Bootstrap (~15 baris)
├── buku/
│   ├── list.html
│   └── tambah.html
├── anggota/
│   ├── list.html
│   └── tambah.html
└── Dokumentasi/        # Folder dokumentasi ini
```

Tidak ada file JavaScript custom yang perlu ditulis — komponen navbar Bootstrap sudah membawa JavaScript-nya sendiri lewat `bootstrap.bundle.min.js` (dijelaskan di [bab 3](#)).

Silakan baca urut dari nomor 1, atau langsung loncat ke bagian yang ingin dipahami.

1. Konsep Dasar Bootstrap

1.1 Apa itu Bootstrap?

Bootstrap adalah **framework CSS** (dan sedikit JavaScript) yang dibuat oleh Twitter, berisi kumpulan siap pakai dari:

- **Utility class** — class CSS kecil dengan satu tugas spesifik, misalnya `.text-center` (rata tengah), `.mb-3` (margin bawah), `.shadow-sm` (bayangan tipis). Alih-alih menulis CSS sendiri di `style.css`, kamu tinggal menempelkan nama class ini ke HTML.
- **Komponen siap pakai** — potongan HTML+CSS+JS yang sudah didesain, seperti navbar, card, tombol, alert, modal, dsb. Kamu tinggal menyalin struktur HTML-nya dan komponen itu langsung berfungsi.
- **Grid system 12 kolom** — sistem tata letak berbasis `.row` dan `.col-*` untuk membagi halaman jadi kolom-kolom responsif (dibahas detail di [bab 4](#)).

Ini adalah kebalikan filosofi dari [jobsheet-03 versi CSS murni](#): di sana **semua** aturan CSS (flexbox navbar, grid kartu, checkbox hack, media query) ditulis sendiri dari nol di `style.css`. Di versi ini, sebagian besar pekerjaan itu sudah "dibungkus" jadi class-class yang tinggal dipakai.

1.2 Cara Memuat Bootstrap: CDN

Jobsheet ini memuat Bootstrap lewat **CDN** (Content Delivery Network) — artinya file CSS dan JS Bootstrap **tidak disimpan** di dalam folder proyek, melainkan diambil langsung dari server publik jsDelivr setiap halaman dibuka:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

Bagian	Artinya
<code><link ... rel="stylesheet"></code>	Memuat file CSS Bootstrap — berisi semua definisi class (<code>.navbar</code> , <code>.card</code> , <code>.btn</code> , dst). Diletakkan di <code><head></code> , sama seperti <code>style.css</code> sendiri.
<code><script src="...bundle.min.js"></code>	Memuat file JavaScript Bootstrap — dibutuhkan untuk komponen yang punya interaksi (buka/tutup navbar mobile, dropdown, modal). Diletakkan di akhir <code><body></code> , setelah semua konten, supaya halaman tampil dulu sebelum skrip dijalankan.
<code>bootstrap@5.3.3</code>	Versi Bootstrap yang dipakai — dikunci di angka <code>5.3.3</code> (bukan <code>@latest</code>) supaya tampilan tidak berubah tiba-tiba kalau Bootstrap merilis versi baru.
<code>.bundle.min.js</code>	Kata <code>bundle</code> berarti file ini sudah menyertakan library Popper.js di dalamnya (dibutuhkan untuk dropdown/tooltip). Kata <code>min</code>

Bagian	Artinya
	berarti sudah diminifikasi (dipadatkan, spasi & komentar dibuang) supaya ukuran file lebih kecil dan cepat diunduh.

Kelebihan CDN: tidak perlu instalasi apa pun (`npm install`, dsb), tinggal tempel dua baris ini dan Bootstrap langsung aktif — cocok untuk belajar dan proyek kecil.

Kekurangannya: halaman butuh **koneksi internet aktif** untuk memuat Bootstrap; kalau CDN sedang down atau internet mati, halaman akan tampil tanpa gaya Bootstrap sama sekali (HTML polos). Untuk proyek produksi skala besar, Bootstrap biasanya di-install lewat `npm` dan disatukan ke proses build sendiri — di luar cakupan jobsheet ini.

1.3 Filosofi Utility-First: Styling Lewat Nama Class

Bandingkan dua cara menulis "judul kartu berwarna biru dengan margin bawah" berikut:

Cara CSS murni (dari [jobsheet-02](#)):

```
section h2 {
  margin-bottom: 1rem;
  color: #1d5b8a;
}
```

```
<h2>Ringkasan</h2>
```

Cara Bootstrap (dipakai di jobsheet ini):

```
<h2 class="card-title mb-3" style="color:#1d5b8a;">Ringkasan</h2>
```

Bootstrap menyediakan `mb-3` (margin-bottom level 3, setara `1rem`) sebagai **utility class siap pakai**, jadi tidak perlu menulis aturan CSS terpisah di file lain. Warna biru `#1d5b8a` di jobsheet ini tetap ditulis lewat atribut `style` langsung karena itu adalah warna brand khusus SIMPUS-Mini yang bukan bagian dari palet warna bawaan Bootstrap — hal ini dibahas lebih lanjut di [bab 6 §6.3](#).

Pola penamaan `mb-3` mengikuti skema **spacing scale** Bootstrap:

Class	Ukuran
<code>mb-0</code>	<code>margin-bottom: 0</code>

Class	Ukuran
mb-1	0.25rem
mb-2	0.5rem
mb-3	1rem
mb-4	1.5rem
mb-5	3rem

Pola yang sama berlaku untuk arah lain: `mt-*` (margin-top), `ms-*` (margin-start/kiri), `me-*` (margin-end/kanan), `p-*` (padding semua sisi), dan seterusnya.

1.4 Breakpoint Bawaan Bootstrap

Alih-alih menulis `@media (max-width: 768px)` sendiri seperti di [jobsheet-03 versi CSS murni](#), Bootstrap sudah punya 6 breakpoint standar yang dipakai konsisten di **semua** class responsifnya (grid, navbar, utility, dst):

Breakpoint	Infix (sisipan nama class)	Lebar Minimal
Extra small	(<i>tanpa infix</i>)	<576px
Small	<code>sm</code>	≥576px
Medium	<code>md</code>	≥768px
Large	<code>lg</code>	≥992px
Extra large	<code>xl</code>	≥1200px
Extra extra large	<code>xxl</code>	≥1400px

Contoh pemakaian: class `col-md-4` (dipakai di [bab 4](#)) berarti "lebar kolom 4/12 bagian, **mulai** dari lebar layar `md` (≥768px) ke atas" — di bawah itu (layar HP), kolom otomatis melebar penuh. Perhatikan arah logikanya **mobile-first**: gaya dasar (tanpa infix) berlaku untuk layar **paling sempit**, lalu class berinfix menambahkan aturan untuk layar yang **lebih lebar** — kebalikan dari strategi *desktop-first* (`max-width`) yang dipakai di [jobsheet-03 versi CSS murni](#).

Karena breakpoint ini sudah "terpasang" di dalam setiap komponen Bootstrap, jobsheet ini **tidak perlu menulis satupun** blok `@media` manual — bandingkan dengan `style.css` jobsheet-03 asli yang punya 2 blok `@media` custom.

Lanjut ke: [Apa yang Berubah di File HTML?](#)

2. Apa yang Berubah di File HTML?

Struktur besar 5 halaman (Beranda, Daftar Buku, Tambah Buku, Daftar Anggota, Tambah Anggota) tetap sama seperti [jobsheet-03 versi CSS murni](#), tapi hampir setiap tag mendapat tambahan atribut `class="..."` berisi nama-nama utility/komponen Bootstrap, dan `<meta viewport>` tetap dipertahankan karena masih wajib untuk responsive design (lihat [dokumentasi jobsheet-03 §1.2](#)).

2.1 Dua Tag Baru di `<head>` dan Akhir `<body>`

```
<head>
...
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
...
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

Urutan **penting**:

1. CSS Bootstrap dimuat **lebih dulu** di `<head>`.
2. `style.css` milik proyek sendiri dimuat **setelah** CSS Bootstrap — supaya kalau ada aturan yang bentrok, override kecil kita ([bab 6](#)) tetap menang (ingat aturan "yang ditulis belakangan menang" dari [dokumentasi jobsheet-03 §1.5](#)).
3. JavaScript Bootstrap diletakkan di **paling akhir** `<body>`, bukan di `<head>` — supaya browser merender seluruh konten HTML terlebih dahulu sebelum mengunduh & menjalankan skrip, membuat halaman terasa lebih cepat tampil.

2.2 `<header>` Diganti Struktur Komponen `.navbar`

Sebelumnya (checkbox hack, lihat [dokumentasi jobsheet-03 bab 2 §2.2](#)):

```
<header>
<h1>SIMPUS-Mini</h1>
```

```

☐
<label for="nav-toggle" class="nav-toggle-label">&#9776;</label>
<nav>...</nav>
</header>

```

Sekarang:

```

<header class="navbar navbar-expand-lg navbar-dark" style="background-color:#1d5b8a;">
  <div class="container">
    <a class="navbar-brand fw-semibold" href="index.html">SIMPUS-Mini</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#l:
      <span class="navbar-toggler-icon"></span>
    </button>
    <nav class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="index.html">Beranda</a></li>
        ...
      </ul>
    </nav>
  </div>
</header>

```

Tidak ada lagi `<input type="checkbox">` — komponen navbar Bootstrap membutuhkan tombol `<button>` sungguhan yang dikontrol lewat atribut `data-bs-*` dan JavaScript Bootstrap, bukan lagi checkbox tersembunyi. Detail lengkap tiap bagiannya dibahas di [bab 3](#).

2.3 `<div class="container">` — Pembatas Lebar Konten

```

<div class="container">
  ...
</div>

```

Ini menggantikan peran `main { max-width: 1000px; margin: 0 auto; }` dari [dokumentasi jobsheet-02](#): class `.container` bawaan Bootstrap otomatis memberi `max-width` berbeda-beda mengikuti **breakpoint aktif** (lihat [bab 1 §1.4](#)), serta padding kiri-kanan dan `margin: 0 auto` untuk menengahkan kontennya — semua dalam **satu** nama class, tanpa CSS tambahan apa pun. Jobsheet ini memakainya baik di dalam `<header>` (membatasi lebar isi navbar) maupun di `<main class="container my-4">` (membatasi lebar konten halaman, `my-4` = margin atas-bawah level 4).

2.4 `<section>` Diganti `<div class="card">`

Sebelumnya:

```

<section>
  <h2>Ringkasan</h2>
  ...
</section>

```

Sekarang:

```

<section class="card shadow-sm mb-4">
  <div class="card-body">
    <h2 class="card-title mb-3" style="color:#1d5b8a;">Ringkasan</h2>
    ...
  </div>
</section>

```

Tag `<section>` (elemen semantic HTML5, tetap dipertahankan sesuai [konsep tag semantic di jobsheet-01](#)) sekarang **juga** diberi class `.card` dari Bootstrap, ditambah `<div class="card-body">` di dalamnya sebagai pembungkus wajib komponen card. Dijelaskan lengkap di [bab 4](#).

2.5 Kartu Statistik: dari `<article>` ke `.row / .col-md-4`

Sebelumnya ([dokumentasi jobsheet-02](#)):

```

<section>
  <h2>Ringkasan</h2>
  <article>
    <h3>Total Buku</h3>
    <p>12</p>
  </article>
  ...
</section>

```

Sekarang:

```

<div class="row g-3 text-center">
  <div class="col-12 col-md-4">
    <div class="p-3 rounded-3" style="background-color:#eef4fa;">
      <h3 class="h6 text-secondary">Total Buku</h3>
      <p class="fs-2 fw-bold mb-0" style="color:#1d5b8a;">12</p>
    </div>
  </div>
  ...
</div>

```

Tag `<article>` diganti `<div>` polos karena perannya sekarang murni sebagai kolom grid Bootstrap (`col-12 col-md-4`), dijelaskan penuh di [bab 4](#). Perhatikan `<h3 class="h6">` — trik

umum Bootstrap untuk memakai tag heading yang **semantically benar** (`h3`, urutan ketiga setelah `h1` navbar dan `h2` judul section) tapi tampil dengan **ukuran font** heading level lain (`h6`, lebih kecil) lewat class utility, tanpa perlu menulis CSS custom untuk itu.

2.6 Tabel: `table-responsive` Tetap Dipakai, Tabelnya Ditambah Class

```
<div class="table-responsive">
  <table class="table table-striped table-hover align-middle">
    <thead style="background-color:#1d5b8a;">
      <tr class="text-white">
        <th>Judul</th>
        ...
```

Pembungkus `<div class="table-responsive">` **tidak berubah** dari [dokumentasi jobsheet-03 §2.3](#) — kebetulan nama class ini identik antara CSS custom jobsheet-03 asli dan Bootstrap, karena keduanya sama-sama menyelesaikan masalah yang sama (scroll horizontal di layar sempit) dengan pola HTML yang sama. Bedanya, class `table-responsive` di sini sudah **didefinisikan** Bootstrap sendiri**, jadi baris `.table-responsive { overflow-x: auto; }` ** di `style.css` (lihat [dokumentasi jobsheet-03 §4.1](#)) tidak perlu ditulis ulang lagi. Detail class tabel lainnya (`table-striped`, `table-hover`) dibahas di [bab 5](#).

2.7 Form: Setiap Isian Dibungkus `<div class="mb-3">`

Sebelumnya (pola `<p>` dari [dokumentasi jobsheet-01 §4.3](#)):

```
<p>
  <label for="judul">Judul</label><br>
  <input type="text" id="judul" name="judul" required>
</p>
```

Sekarang:

```
<div class="mb-3">
  <label for="judul" class="form-label fw-semibold">Judul</label>
  <input type="text" class="form-control" id="judul" name="judul" required>
</div>
```

Pembungkus `<p>` diganti `<div class="mb-3">` — secara semantic, sebuah isian form (label + input) bukan benar-benar sebuah "paragraf teks", jadi `<div>` polos yang diberi class utility spacing (`mb-3`) lebih tepat, sekaligus konvensi resmi yang dipakai dokumentasi Bootstrap sendiri untuk form. Atribut `for` / `id` yang menghubungkan label ke input ([konsepnya dari jobsheet-01](#)) tetap **tidak berubah** — Bootstrap tidak mengubah perilaku HTML native ini sama

sekali, hanya menambah tampilan lewat class `form-label` dan `form-control`. Detail lengkap dibahas di [bab 5](#).

Lanjut ke: [Navbar Responsif ala Bootstrap](#)

3. Navbar Responsif ala Bootstrap

Bab ini membedah bagian yang paling banyak berubah dari [versi CSS murni](#): menu hamburger yang dulunya murni CSS (checkbox hack) sekarang memakai komponen `.navbar` bawaan Bootstrap yang **membutuhkan JavaScript** untuk animasi buka-tutupnya.

3.1 Struktur HTML Lengkap

```
<header class="navbar navbar-expand-lg navbar-dark" style="background-color:#1d5b8a;">
  <div class="container">
    <a class="navbar-brand fw-semibold" href="index.html">SIMPUS-Mini</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#1
      <span class="navbar-toggler-icon"></span>
    </button>
    <nav class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="index.html">Beranda</a></li>
        <li class="nav-item"><a class="nav-link" href="buku/list.html">Daftar Buku</a></li>
        ...
      </ul>
    </nav>
  </div>
</header>
```

3.2 Class `.navbar`, `.navbar-expand-lg`, `.navbar-dark`

Class	Fungsi
<code>.navbar</code>	Menandai elemen ini sebagai komponen navbar Bootstrap — mengaktifkan seluruh perilaku dasarnya (flexbox internal, padding, dst).
<code>.navbar-expand-lg</code>	Kunci responsivitasnya. Berarti: navbar tampil horizontal terbuka (seperti desktop) mulai breakpoint <code>lg</code> ($\geq 992\text{px}$, lihat bab 1 §1.4) ke atas. Di bawah <code>lg</code> (tablet & HP), navbar otomatis "terlipat" jadi tombol hamburger.

Class	Fungsi
<code>.navbar-dark</code>	Mengatur skema warna teks/ikon navbar supaya kontras di atas latar gelap (teks putih, dsb) — cocok dipakai bersama <code>style="background-color:#1d5b8a;"</code> (biru tua) yang ditulis manual karena warna brand ini bukan salah satu tema warna bawaan Bootstrap.

Bandingkan dengan pendekatan CSS murni yang hanya punya **satu** breakpoint hamburger (480px, lihat [dokumentasi jobsheet-03 §5.3](#)) — di jobsheet ini, breakpoint kapan navbar "terlipat" bisa diganti hanya dengan mengubah infix di nama class (`navbar-expand-md`, `navbar-expand-sm`, dst), tanpa menyentuh CSS sama sekali.

3.3 Tombol Hamburger: `.navbar-toggler`

```
<button class="navbar-toggler" type="button"
  data-bs-toggle="collapse"
  data-bs-target="#navMenu"
  aria-controls="navMenu"
  aria-expanded="false"
  aria-label="Toggle navigation">
  <span class="navbar-toggler-icon"></span>
</button>
```

Ini adalah **tombol asli** (`<button>`), bukan `<label>` yang berpura-pura jadi tombol seperti di [checkbox hack versi CSS murni](#). Beberapa atribut pentingnya:

Atribut	Fungsi
<code>data-bs-toggle="collapse"</code>	Memberi tahu JavaScript Bootstrap: "tombol ini mengontrol komponen collapse (buka/tutup)". Ini adalah data attribute — atribut HTML custom berawalan <code>data-</code> yang boleh dibaca skrip, konsepnya mirip <code>id/class</code> tapi khusus dipakai sebagai "pengait" JavaScript, bukan untuk styling CSS.
<code>data-bs-target="#navMenu"</code>	Menentukan elemen mana yang dibuka/ditutup ketika tombol ini diklik — mengacu ke <code>id="navMenu"</code> pada <code><nav></code> di bawahnya. Konsepnya mirip <code>for="nav-toggle"</code> pada checkbox hack yang menghubungkan label ke checkbox lewat <code>id</code> — bedanya di sini yang membaca hubungan <code>id</code> ini adalah JavaScript , bukan CSS.
<code>aria-controls</code> , <code>aria-expanded</code> , <code>aria-label</code>	Atribut aksesibilitas (accessibility) untuk pembaca layar (screen reader) — menjelaskan tombol ini mengontrol elemen apa dan

Atribut	Fungsi
	statusnya sedang terbuka/tertutup. Bootstrap otomatis mengubah nilai <code>aria-expanded</code> lewat JavaScript saat tombol diklik.
<code></code>	Elemen kosong yang diberi gaya CSS bawaan Bootstrap berupa ikon garis tiga (≡) memakai <code>background-image</code> — pengganti karakter HTML entity <code>&#9776;</code> yang dipakai manual di versi CSS murni .

3.4 Elemen yang Dibuka-Tutup: `.collapse.navbar-collapse`

```
<nav class="collapse navbar-collapse" id="navMenu">
  <ul class="navbar-nav ms-auto">
    ...
  </ul>
</nav>
```

- `.collapse` — class umum Bootstrap untuk elemen yang bisa disembunyikan/ditampilkan dengan animasi geser (slide). Secara default elemen ini **tersembunyi** kalau ukuran layar di bawah breakpoint `.navbar-expand-lg`.
- `.navbar-collapse` — varian khusus `.collapse` yang disesuaikan untuk konteks navbar (misalnya otomatis kembali terlihat horizontal begitu layar $\geq$ breakpoint `lg`, tanpa perlu status "terbuka/tertutup" sama sekali di layar besar).
- `id="navMenu"` — inilah target yang dicari `data-bs-target="#navMenu"` dari tombol di [§3.3](#).
- `.navbar-nav` pada `<ul>` — mengubah daftar `<li>` di dalamnya jadi item navbar dengan spacing dan layout flexbox bawaan Bootstrap (menggantikan `header nav ul { display: flex; gap: 1.25rem; }` manual dari [dokumentasi jobsheet-02](#)).
- `.ms-auto` (*margin-start: auto*) — mendorong seluruh menu ke **sisi kanan**** navbar, menggantikan `justify-content: space-between` pada** `<header>` dari CSS murni.

3.5 Kenapa Ini Butuh JavaScript, Sedangkan Checkbox Hack Tidak?

Perbedaan filosofis penting:

- Checkbox hack** ([dokumentasi jobsheet-03 bab 3](#)) memanfaatkan pseudo-class `:checked` bawaan CSS pada elemen `<input type="checkbox">` — **tidak butuh JavaScript sama sekali**, tapi triknya cukup "curang" (meminjam perilaku checkbox untuk keperluan di luar konteks form) dan terbatas kemampuannya (tidak ada animasi transisi halus, sulit dikombinasikan dengan komponen lain).

- **Komponen** `.collapse` **Bootstrap** memakai JavaScript sungguhan (`bootstrap.bundle.min.js`, dimuat di [bab 2 §2.1](#)) yang menambah/menghapus class (misalnya `.show`) dan attribute style secara dinamis saat tombol diklik, sekaligus menjalankan animasi CSS transition untuk efek geser yang halus. Ini lebih "resmi" dan fleksibel, tapi **bergantung pada file JavaScript berhasil dimuat** — kalau CDN gagal diakses ([bab 1 §1.2](#)), tombol hamburger tidak akan berfungsi sama sekali.

Ini adalah trade-off nyata antara dua pendekatan: **CSS murni** = tidak ada dependensi eksternal tapi teknik lebih rumit ditulis sendiri; **Bootstrap** = jauh lebih cepat dibangun & lebih banyak fitur, tapi bergantung pada library pihak ketiga.

3.6 Cara Mencoba Sendiri

1. Buka `index.html` di browser, lebarkan jendela browser sampai lebih dari 992px — menu navigasi akan tampil sejajar horizontal (efek `.navbar-expand-lg`).
2. Sempitkan jendela browser (atau buka DevTools mode responsif, `Ctrl+Shift+M`) sampai di bawah 992px — menu berubah jadi ikon ☰ di kanan atas.
3. Klik ikon tersebut — perhatikan menu muncul dengan **animasi geser** halus** dari atas, berbeda dengan checkbox hack yang muncul/hilang** secara instan (`display: none` → `display: block` tidak beranimasi).
4. Buka tab **Network** di DevTools, refresh halaman, cari `bootstrap.bundle.min.js` — pastikan statusnya berhasil dimuat (200), sebagai bukti bahwa interaksi ini memang bergantung pada file JavaScript tersebut.

Lanjut ke: [Grid System & Komponen Card](#)

4. Grid System & Komponen Card

Bab ini membahas penggantian dua hal dari [jobsheet-02](#): CSS Grid manual (`display: grid; grid-template-columns: repeat(3, 1fr);`) diganti **grid system 12 kolom Bootstrap**, dan `<section>` polos diganti komponen `.card`.

4.1 Kode HTML Lengkap (Kartu Statistik di Beranda)

```
<section class="card shadow-sm mb-4">
  <div class="card-body">
    <h2 class="card-title mb-3" style="color:#1d5b8a;">Ringkasan</h2>
    <div class="row g-3 text-center">
      <div class="col-12 col-md-4">
        <div class="p-3 rounded-3" style="background-color:#eef4fa;">
          <h3 class="h6 text-secondary">Total Buku</h3>
          <p class="fs-2 fw-bold mb-0" style="color:#1d5b8a;">12</p>
```

```

        </div>
    </div>
    <div class="col-12 col-md-4"> ... Total Anggota ... </div>
    <div class="col-12 col-md-4"> ... Sedang Dipinjam ... </div>
</div>
</div>
</section>

```

4.2 Komponen Card: `.card`, `.card-body`, `.card-title`

Class	Fungsi
<code>.card</code>	Bingkai kotak dasar: border tipis, sudut membulat (<code>border-radius</code>), latar putih. Menggantikan CSS custom <code>section { border-radius: 8px; box-shadow: ...; }</code> dari dokumentasi jobsheet-02 §5.3 .
<code>.card-body</code>	Pembungkus wajib di dalam <code>.card</code> yang memberi padding dalam. Tanpa ini, isi kartu akan menempel langsung ke tepi border.
<code>.card-title</code>	Ditempelkan ke heading (di sini <code><h2></code>) di dalam card — memberi sedikit gaya khas judul kartu (margin bawah otomatis, dsb).
<code>.shadow-sm</code>	Bayangan halus di sekeliling card — pengganti <code>box-shadow: 0 1px 3px rgba(0,0,0,0.08);</code> custom dari CSS murni. Bootstrap menyediakan beberapa level: <code>.shadow-sm</code> (tipis), <code>.shadow</code> (sedang), <code>.shadow-lg</code> (tebal).
<code>.mb-4</code>	Jarak margin bawah antar-card, menggantikan <code>section { margin-bottom: 1.5rem; }</code> .

Perhatikan pola ini **berulang** di semua halaman lain (`buku/list.html`, `anggota/tambah.html`, dst) — setiap `<section>` konten utama dibungkus `.card + .card-body` dengan pola yang identik, memberi tampilan konsisten di seluruh aplikasi tanpa perlu mengulang definisi CSS.

4.3 Grid 12 Kolom: `.row` dan `.col-*`

Konsep dasar grid system Bootstrap: setiap baris (`.row`) dibagi menjadi **12 kolom virtual** sama lebar. Class `.col-*` menentukan **berapa dari 12 kolom itu**** yang dipakai suatu elemen.**

```

<div class="row g-3 text-center">
  <div class="col-12 col-md-4"> ... </div>
  <div class="col-12 col-md-4"> ... </div>

```

```
<div class="col-12 col-md-4"> ... </div>
</div>
```

Class	Artinya
<code>.row</code>	Wadah (container) untuk kolom-kolom di dalamnya — wajib ada sebagai pembungkus setiap kumpulan <code>.col-*</code> . Secara internal <code>display: flex</code> dengan sedikit negative margin untuk kompensasi padding kolom.
<code>.col-12</code>	Di layar paling sempit (tanpa infix breakpoint, <code><768px</code>), kolom ini memakai 12 dari 12 bagian — artinya melebar penuh satu baris sendiri.
<code>.col-md-4</code>	Mulai breakpoint <code>md</code> ($\geq 768px$, lihat bab 1 §1.4) ke atas, kolom ini memakai 4 dari 12 bagian — karena ada 3 kartu dan $4 \times 3 = 12$, ketiganya pas sejajar dalam satu baris penuh.
<code>.g-3</code>	<i>Gutter</i> (jarak antar kolom) level 3 (<code>1rem</code>) — mengatur jarak horizontal dan vertikal antar kartu sekaligus, termasuk saat kartu ketiga turun ke baris baru di layar sempit.
<code>.text-center</code>	Utility class perataan teks ke tengah, diterapkan ke seluruh <code>.row</code> sehingga berlaku ke semua kolom di dalamnya.

Progresi responsifnya (mobile-first, dari [bab 1 §1.4](#)):

Lebar Layar	Class Aktif	Susunan Kartu
<code><768px</code> (HP)	<code>.col-12</code>	1 kolom, bertumpuk vertikal
$\geq 768px$ (tablet/desktop)	<code>.col-md-4</code>	3 kolom sejajar

Dibandingkan [versi CSS murni](#) yang punya **3 tingkat** (3 kolom desktop → 2 kolom tablet → 1 kolom mobile lewat 2 blok `@media` terpisah), versi Bootstrap ini disederhanakan jadi **2 tingkat** saja (`col-12` lalu langsung `col-md-4`) — kamu bisa menambah tingkat ketiga sendiri sebagai latihan, misalnya menyisipkan `col-sm-6` di antaranya supaya ada juga tampilan 2-kolom di breakpoint `sm` (lihat [bab 6 §6.4](#)).

4.4 Styling Isi Tiap Kartu Statistik

```
<div class="p-3 rounded-3" style="background-color:#eef4fa;">
  <h3 class="h6 text-secondary">Total Buku</h3>
```

```
<p class="fs-2 fw-bold mb-0" style="color:#1d5b8a;">12</p>
</div>
```

Class	Fungsi
<code>.p-3</code>	Padding semua sisi level 3 (<code>1rem</code>) — menggantikan <code>article { padding: 1.25rem; }</code> .
<code>.rounded-3</code>	Sudut membulat level 3 — menggantikan <code>article { border-radius: 8px; }</code> .
<code>.h6</code> (pada tag <code><h3></code>)	Trik "ukuran heading berbeda dari levelnya" yang sudah disinggung di bab 2 §2.5 — tag tetap <code><h3></code> demi urutan heading semantic yang benar, tapi tampilan fontnya memakai ukuran <code><h6></code> yang lebih kecil, cocok untuk label kecil "Total Buku" dst.
<code>.text-secondary</code>	Warna teks abu-abu bawaan tema Bootstrap — menggantikan <code>color: #55677a; custom.</code>
<code>.fs-2</code> (<i>font-size 2</i>)	Skala ukuran font bawaan Bootstrap (dari <code>.fs-1</code> terbesar sampai <code>.fs-6</code> terkecil) — dipakai untuk angka besar statistik, menggantikan <code>font-size: 1.8rem; custom.</code>
<code>.fw-bold</code>	<i>Font-weight bold</i> — menggantikan <code>font-weight: 700; .</code>
<code>.mb-0</code>	Menghapus margin bawah default paragraf <code><p></code> supaya angka statistik menempel rapat ke label di atasnya.

Satu-satunya nilai yang **tetap** ditulis manual lewat atribut `style` adalah warna latar `#eef4fa` dan warna teks `#1d5b8a` — karena keduanya adalah warna **brand khusus** SIMPUS-Mini, bukan bagian dari palet warna tema bawaan Bootstrap (`primary`, `secondary`, `success`, dst). Ini dibahas lebih dalam di [bab 6 §6.3](#).

Lanjut ke: [Tabel & Form dengan Utility Class Bootstrap](#)

5. Tabel & Form dengan Utility Class Bootstrap

5.1 Tabel: `.table`, `.table-striped`, `.table-hover`

Kode dari `buku/list.html` dan `anggota/list.html`:

```

<div class="table-responsive">
  <table class="table table-striped table-hover align-middle">
    <thead style="background-color:#1d5b8a;">
      <tr class="text-white">
        <th>Judul</th>
        <th>Pengarang</th>
        <th>Tahun</th>
        <th>Stok</th>
        <th>Aksi</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Laskar Pelangi</td>
        ...
        <td>
          <button type="button" class="btn btn-warning btn-sm text-white">Edit</button>
          <button type="button" class="btn btn-danger btn-sm">Hapus</button>
        </td>
      </tr>
      ...
    </tbody>
  </table>
</div>

```

Class	Fungsi	Menggantikan (CSS Murni)
<code>.table</code>	Class dasar wajib — memberi spacing sel, border tipis antar baris, dan lebar penuh (<code>width: 100%</code>).	<code>table { width: 100%; border-collapse: collapse; } th, td { padding: ...; border-bottom: ...; }</code>
<code>.table-striped</code>	Baris genap-ganjil otomatis diberi warna latar berselang-seling.	<code>tbody tr:nth-child(even) { background-color: #f7f9fb; }</code>
<code>.table-hover</code>	Baris berubah warna latar saat kursor mouse diarahkan ke atasnya.	<code>tbody tr:hover { background-color: #eef4fa; }</code>
<code>.align-middle</code>	Isi setiap sel (<code><td></code>) rata tengah secara vertikal — berguna khususnya di kolom "Aksi" supaya tombol Edit/Hapus sejajar rapi dengan teks di kolom lain.	<i>(tidak ada di versi CSS murni)</i>

Class	Fungsi	Menggantikan (CSS Murni)
<code>.table-responsive</code> (pada <code><div></code> pembungkus)	Scroll horizontal otomatis kalau tabel lebih lebar dari layar.	<code>.table-responsive { overflow-x: auto; }</code> — nama classnya kebetulan identik, lihat dokumentasi jobsheet-03 §2.3 .

Warna header tabel (`background-color:#1d5b8a`) dan teks putihnya (`.text-white` pada `<tr>`) tetap ditulis manual dengan alasan yang sama seperti [bab 4 §4.4](#): warna brand khusus, bukan bagian tema bawaan Bootstrap.

5.2 Tombol Aksi: `.btn`, `.btn-warning`, `.btn-danger`, `.btn-sm`

```
<button type="button" class="btn btn-warning btn-sm text-white">Edit</button>
<button type="button" class="btn btn-danger btn-sm">Hapus</button>
```

Bootstrap menyediakan sistem warna tombol **semantik** (berdasarkan makna/fungsi, bukan sekadar warna):

Class	Warna Bawaan	Dipakai untuk
<code>.btn-primary</code>	Biru	Aksi utama (tidak dipakai di jobsheet ini karena warna brand SIMPUS-Mini berbeda dari biru default Bootstrap)
<code>.btn-warning</code>	Kuning/oranye	Tombol Edit — menandakan "hati-hati, akan mengubah data"
<code>.btn-danger</code>	Merah	Tombol Hapus — menandakan aksi destruktif
<code>.btn-sm</code>	(<i>bukan warna</i>)	Memperkecil ukuran tombol (padding & font lebih kecil), cocok untuk tombol di dalam sel tabel yang sempit

Dua class ini menggantikan CSS custom:

```
td button:first-of-type { background-color: #f0ad4e; color: #fff; }
td button:last-of-type { background-color: #d9534f; color: #fff; }
```

dari [dokumentasi jobsheet-02](#). Selector `:first-of-type / :last-of-type` yang **bergantung urutan** tombol di HTML (rawan salah kalau urutan tombol berubah) digantikan class yang **eksplisit menempel** ke tombol yang dimaksud – pendekatan Bootstrap ini lebih tahan terhadap perubahan struktur HTML di kemudian hari. Tambahkan `.text-white` pada tombol Edit karena warna default teks `.btn-warning` adalah gelap (agar kontras dengan latar kuning terangnya), sedangkan jobsheet ini tetap mengikuti gaya teks putih dari [versi CSS murni](#).

5.3 Form: `.form-label`, `.form-control`, `.form-select`

Kode dari `buku/tambah.html`:

```
<form>
  <div class="mb-3">
    <label for="judul" class="form-label fw-semibold">Judul</label>
    <input type="text" class="form-control" id="judul" name="judul" required>
  </div>
  ...
  <div class="mb-3">
    <label for="kategori" class="form-label fw-semibold">Kategori</label>
    <select class="form-select" id="kategori" name="kategori">
      <option value="fiksi">Fiksi</option>
      ...
    </select>
  </div>
  <button type="submit" class="btn" style="background-color:#1d5b8a; color:#fff;">Simpan</button>
</form>
```

Class	Ditempelkan ke	Fungsi	Menggantikan
<code>.form-label</code>	<code><label></code>	Jarak margin bawah label yang konsisten dan sedikit penyesuaian tampilan.	<code>form label { display: block; margin-bottom: 0.35rem; }</code>
<code>.fw-semibold</code>	<code><label></code>	Ketebalan font semi-bold, agar label tetap menonjol.	<code>form label { font-weight: 600; color: #444; }</code>
<code>.form-control</code>	<code><input></code>	Lebar penuh, padding nyaman, border, dan efek fokus (garis biru saat elemen diklik) – semuanya otomatis.	<code>form input { width: 100%; padding: 0.55rem 0.7rem; border: 1px solid #cdd4da; border-radius: 4px; }</code>

Class	Ditempelkan ke	Fungsi	Menggantikan
<code>.form-select</code>	<code><select></code>	Sama seperti <code>.form-control</code> tapi khusus dropdown, termasuk ikon panah bawah otomatis.	<code>form select { ... }</code> (aturan sama dengan input di CSS murni)

Perhatikan atribut `for`, `id`, `name`, `type`, `required`, `min`, `max` pada setiap input **tidak berubah sama sekali** dari [versi CSS murni](#) — semua itu adalah perilaku HTML native (validasi form bawaan browser, hubungan label-input) yang **sepenuhnya independen** dari framework CSS apa pun yang dipakai. Bootstrap hanya menambah **tampilan** lewat class, tidak pernah mengubah cara kerja HTML form itu sendiri.

Tombol submit `<button type="submit" class="btn" style="background-color:#1d5b8a; color:#fff;">` memakai class dasar `.btn` saja (bukan `.btn-primary` dkk) karena warna birunya adalah warna brand khusus yang ditulis manual — pola yang sama seperti dibahas di [bab 4 §4.4](#). Efek hover-nya (`background-color: #164869`) didefinisikan di `style.css` (dibahas di [bab 6](#)) karena Bootstrap tidak tahu warna hover yang tepat untuk warna brand custom ini.

Lanjut ke: [Rangkuman & Perbandingan dengan CSS Murni](#)

6. Rangkuman & Perbandingan dengan CSS Murni

6.1 Rangkuman Keseluruhan

Bagian	Lokasi Kode	Konsep yang Dipelajari
Konsep Dasar Bootstrap	<code><link>/<script></code> CDN	Framework CSS, utility-first, CDN, breakpoint bawaan
Perubahan HTML	Semua file <code>.html</code>	<code>.container</code> , <code>.card</code> , pola pembungkus form <code>.mb-3</code>
Navbar Responsif	<code>.navbar</code> , <code>.navbar-toggler</code> , <code>.collapse</code>	Komponen berbasis JavaScript, atribut <code>data-bs-*</code> , aksesibilitas (<code>aria-*</code>)

Bagian	Lokasi Kode	Konsep yang Dipelajari
Grid & Card	<code>.row</code> , <code>.col-*</code> , <code>.card</code>	Grid 12 kolom, pendekatan mobile-first Bootstrap
Tabel & Form	<code>.table-*</code> , <code>.btn-*</code> , <code>.form-control</code>	Utility class semantik (warna berdasarkan makna), styling form tanpa CSS custom

6.2 Tabel Perbandingan Class Bootstrap vs CSS Murni

Ringkasan seluruh pemetaan yang sudah dibahas di bab-bab sebelumnya:

Kebutuhan	CSS Murni (jobsheet-03 asli)	Bootstrap (jobsheet ini)
Membatasi & menengahkan lebar konten	<code>main { max-width: 1000px; margin: 0 auto; }</code>	<code>.container</code>
Kartu putih dengan bayangan	<code>section { border-radius: 8px; box-shadow: ...; }</code>	<code>.card</code> + <code>.shadow-sm</code>
Grid 3 kolom kartu statistik	<code>display: grid; grid-template-columns: repeat(3, 1fr);</code>	<code>.row</code> + <code>.col-md-4</code>
Navbar hamburger	Checkbox hack (<code>:checked</code> + <code>~</code>)	<code>.navbar-toggler</code> + <code>.collapse</code> (JS)
Tabel belang & hover	<code>nth-child(even)</code> , <code>:hover</code>	<code>.table-striped</code> , <code>.table-hover</code>
Tabel scroll horizontal	<code>.table-responsive { overflow-x: auto; }</code> (custom)	<code>.table-responsive</code> (bawaan, nama sama)
Tombol warna	<code>td button:first-of-type { background: ...; }</code>	<code>.btn-warning</code> , <code>.btn-danger</code>
Input & select form	<code>form input { width:100%; padding:...; border:...; }</code>	<code>.form-control</code> , <code>.form-select</code>
Breakpoint tablet/mobile	<code>@media (max-width: 768px) / (max-width: 480px)</code> custom	Infix bawaan: <code>sm</code> , <code>md</code> , <code>lg</code> , <code>xl</code> , <code>xxl</code>

Kebutuhan	CSS Murni (jobsheet-03 asli)	Bootstrap (jobsheet ini)
Baris CSS custom dibutuhkan	~245 baris	~15 baris

6.3 Kapan Tetap Perlu CSS Custom?

Meski Bootstrap dipakai di hampir seluruh halaman, jobsheet ini **tidak sepenuhnya bebas** dari CSS/atribut `style` manual. Perhatikan warna `#1d5b8a` (biru SIMPUS-Mini) muncul berkali-kali lewat atribut `style="..."` langsung di HTML ([bab 4 §4.4](#), [bab 5 §5.1](#)), dan file [assets/css/style.css](#) masih berisi beberapa baris:

```
.navbar-brand,
.nav-link {
  color: #fff !important;
}

.nav-link:hover,
.nav-link.active {
  color: #d9e8f5 !important;
}

form button[type="submit"]:hover,
button[type="submit"]:hover {
  background-color: #164869 !important;
}
```

Ini terjadi karena **palet warna tema bawaan Bootstrap** (`primary` = biru `#0d6efd`, `secondary` = abu-abu, dst) **berbeda** dari warna brand SIMPUS-Mini (`#1d5b8a`) yang sudah ditentukan sejak [jobsheet-01](#). Ada dua pilihan setiap kali ini terjadi:

1. **Pakai warna tema bawaan Bootstrap apa adanya** (`.btn-primary`, `.bg-primary`, dst) — paling cepat dikerjakan, tapi warnanya jadi biru default Bootstrap, bukan biru brand SIMPUS-Mini.
2. **Tetap pakai warna brand custom** lewat atribut `style` atau sedikit CSS tambahan — seperti yang dipilih jobsheet ini, demi menjaga identitas visual tetap sama dengan [jobsheet-03 versi CSS murni](#).

Perhatikan tanda seru (`!important`) pada 3 aturan di atas — dipakai karena Bootstrap sendiri sudah mendefinisikan warna default untuk `.nav-link` dkk dengan spesifisitas tertentu; `!important` diperlukan di sini supaya warna brand custom benar-benar menang menimpa aturan bawaan Bootstrap tersebut. Ini konsekuensi umum ketika mengombinasikan warna custom di atas framework siap pakai — pemakaian `!important` yang berlebihan sebenarnya

sering dianggap "bau kode" (code smell) karena menyulitkan override berikutnya, jadi sebaiknya dipakai secukupnya saja seperti di sini, bukan di semua tempat.

Kesimpulannya: Bootstrap sangat mempercepat pengerjaan tata letak dan komponen umum, tapi begitu ada kebutuhan **desain brand yang** spesifik** (warna, font khusus, dst di luar tema bawaan), tetap** dibutuhkan sedikit CSS custom untuk menyesuaikannya — Bootstrap menggantikan **sebagian besar** pekerjaan CSS, bukan seluruhnya.

6.4 Ide Latihan Tambahan (Opsional)

1. **Ganti warna brand ke tema bawaan Bootstrap** — hapus semua `style="background-color:#1d5b8a;"` dan `style="color:#1d5b8a;"`, ganti dengan class bawaan seperti `.bg-primary` / `.text-primary`, lalu bandingkan seberapa banyak baris `style.css` yang jadi tidak diperlukan lagi.
2. **Tambah breakpoint ketiga** di grid kartu statistik — sisipkan `col-sm-6` di antara `col-12` dan `col-md-4` ([bab 4 §4.3](#)) supaya ada juga tampilan 2 kolom di breakpoint `sm`, meniru progresi 3 tingkat dari [versi CSS murni](#).
3. **Ganti breakpoint navbar** dari `.navbar-expand-lg` menjadi `.navbar-expand-md`, amati di lebar layar berapa navbar mulai "terlipat" jadi hamburger — buktikan bahwa breakpoint Bootstrap bisa diganti hanya lewat nama class, tanpa CSS tambahan sama sekali.
4. **Tambahkan komponen Bootstrap baru** yang belum dipakai jobsheet ini, misalnya [.badge](#) untuk menandai status "Tersedia"/"Kosong" di kolom Stok pada `buku/list.html`, atau [.alert](#) untuk menampilkan pesan sukses setelah form disimpan.
5. **Bandingkan ukuran file** — buka DevTools tab **Network**, refresh `index.html` versi Bootstrap ini dan bandingkan total ukuran yang diunduh (termasuk CSS+JS Bootstrap dari CDN) dengan versi [jobsheet-03 CSS murni](#) yang hanya memuat satu file `style.css` kecil — diskusikan trade-off ukuran unduhan vs kecepatan pengembangan.

Kalau ada bagian yang masih membingungkan, terutama soal grid 12 kolom atau atribut `data-bs-*`, coba baca ulang [bab 3](#) dan [bab 4](#) sambil membuka [dokumentasi resmi Bootstrap 5.3](#) di tab browser terpisah untuk menjelajahi komponen lain yang tersedia.